Modern C# Language Enhancements (C# 6 to C# 10)

---

1. OUT Parameters Enhancements (C# 7.0)

1.1 Traditional OUT Parameters (Before C# 7)

OUT parameters require pre-declaring variables:

string name;

GetDetails(out name);

1.2 Inline OUT Variable Declaration

C# 7 allows declaration inside the call:

GetDetails(out string name);

Benefits

- Cleaner syntax
- Fewer lines
- More readable

1.3 Using var with OUT

Type inference is supported:

GetDetails(out var name);

1.4 Scope Behavior

The variable exists inside the surrounding block only.

1.5 Ignoring OUT Values

Use discard _:

GetDetails(out var name, out _);

1.6 TryParse Pattern — Real-World Usage

bool ok = int.TryParse("10", out int x);

Purpose

- Safely parse without exceptions
- Returns success status + parsed value

---

2. Pattern Matching (C# 7 → C# 10)

Pattern matching provides structured checking + value extraction.

2.1 is-Pattern

if (shape is Circle c)

```
  Console.WriteLine(c.Radius);
```

Type check and cast in one operation.

2.2 switch Pattern Matching

```
switch(shape)
{
  case Circle c:
    ...
    break;
  case Rectangle r:
    ...
    break;
}
```

Cleaner than multiple if-else chains.

2.3 when Clause

```
case Rectangle r when r.Length == r.Height:
  Console.WriteLine("Square");
```

Notes:

- Adds condition logic
- Order matters
- default runs last

2.4 Property & Nested Property Patterns

C# 9

```
if (dev is Developer { Manager: { FirstName: { Length: 6 } } })
```

C# 10

```
if (dev is Developer { Manager.FirstName.Length: 6 })
```

Outcome

- Natural reading style
- Cleaner syntax

---

3. Ref Locals & Ref Returns (C# 7.0)

3.1 Ref Local

Stores **reference**, not copy:

```
ref int b = ref a;
b = 5;   // a becomes 5
```

3.2 Ref Return

A method returns a reference:

ref int Find(...)

Primary Benefits

- Performance optimization
- In-place modification

---

4. Tuples & ValueTuples (C# 7+)

4.1 Old Tuple (Class-Based — Not Recommended)

Issues:

- Reference type (heap allocation)
- Poor naming (Item1, Item2)
- Max 8 items

4.2 ValueTuple (Preferred)

(int count, double sum) result = GetData();

Or:

var result = GetData();

return (count, sum);

Advantages

- Value type (stack)
- Named elements
- Faster
- More readable

4.3 Tuple Deconstruction

var (count, sum) = GetData();

---

5. Local Functions (C# 7+)

5.1 Definition

Methods declared **inside another method**.

Use Cases

- Helper logic
- Validation
- Recursion
- Iterator logic

Example:

```
void Main()
{
   bool Validate() => ...;
}
```

Rules

- No access modifiers
- Not overloadable
- Only usable inside parent method
- Can access outer variables

5.2 Static Local Functions (C# 8)

```
static int Add(int a, int b) => a + b;
```

Prevents access to external variables — improves safety.

---

6. Throw Expressions (C# 7)

Allows throwing exceptions inside expressions:

```
string name = value ?? throw new ArgumentNullException();
```

Also works in:

- conditional expressions
- expression-bodied members
- property initializers

---

7. String Interpolation (C# 6)

```
$"Name: {name}, Salary: {salary}"
```

Replaces string.Format() with cleaner syntax.

---

8. Auto-Property Enhancements (C# 6 → 9)

8.1 Auto Property Initializer

```
public List<Order> Orders { get; set; } = new List<Order>();
```

8.2 Init-Only Setters (C# 9)

Used for immutability:

```
public int Year { get; init; }
```

Usage:

```
var p = new Person { Year = 1999 };
```

p.Year = 2020; // ERROR

---

9. Target-Typed new() (C# 9)

List<int> numbers = new();

Type inferred from left-hand side.

---

10. File-Scoped Namespaces (C# 10)

Old Style

namespace A {

  class B {}

}

New Style

namespace A;

class B {}

Removes braces → cleaner files.

---

11. Global Using Directives (C# 10)

Define once:

global using System;

global using System.Collections.Generic;

Becomes available globally → reduces duplication.

---

12. Natural Lambda Types (C# 10)

var upper = (string s) => s.ToUpper();

Compiler infers delegate automatically.

If ambiguous, specify type.

---

13. Indices & Ranges (C# 8)

int[] a = {0,1,2,3,4,5};

var last = a[^1];   // 5

var range = a[2..5]; // 2,3,4

Supports slicing and reverse indexing.

---

14. Object Deconstruction Support

Objects can return multiple values:

(var f, var l) = person;

Define:

void Deconstruct(out string f, out string l)

---

15. Performance-Oriented Features (Advanced)

Span

Efficient memory slices without copying.

stackalloc

Stack-allocated memory blocks.

ref struct

Ensures type stays on stack only.

readonly struct

Prevents modification.

ref struct + using Support

Allows deterministic disposal.

Use Cases

- High-performance systems
- Low-level memory optimization

---

16. Version-Wise Summary

C# 6

- String interpolation
- Auto-property initializers

C# 7

- OUT variable improvements
- Pattern matching
- Local functions
- Ref locals & returns
- Throw expressions

- ValueTuple

C# 8

- Static local functions
- Indices & ranges
- Improved readonly and ref struct support

C# 9

- Init-only setters
- Target-typed new
- Record types
- Advanced pattern matching

C# 10

- File-scoped namespace
- Global usings
- Natural lambda types
- Extended property patterns

---

17. Quick Exam-Ready Revision Points

- out var allowed
- Ignore OUT using _
- TryParse uses OUT to return conversion + status
- Pattern matching available in:
    - is
    - switch
    - when
    - property patterns
- Local functions support recursion
- init enforces immutability
- ValueTuple replaces old class-based Tuple
- ref locals modify original objects
- Natural lambdas infer delegate type
- ^ = index from end
- .. = range slicing

- File-scoped namespace cleans syntax
- Global usings remove repetitive imports

Conclusion

These enhancements collectively modernize C#, enabling:

- More expressive syntax
- Safer immutable designs
- Cleaner high-level code
- High-performance memory operations
- Reduced boilerplate
- Stronger type inference

They allow developers to write **clearer, faster, safer, and more maintainable code**, aligned with modern programming paradigms.
